

Exploring the State Space of Districting Small Grid Graphs

A background reading guide for your summer project

Welcome! This document is meant to bring you up to speed on the mathematical and computational background for our summer project. The goal is that by the end you'll understand (1) why anyone cares about sampling districting plans, (2) how the Markov chains we use actually work, (3) what our `gmstrat` pipeline is doing under the hood, and (4) what it concretely means to "explore the state space" of districtings of an $m \times n$ grid.

1. The big picture: redistricting as a graph problem

Every ten years, after the U.S. Census, each state redraws the boundaries of its congressional and state legislative districts. Boundaries matter enormously — the same set of voters can produce very different election outcomes depending on how they're grouped. When boundaries are drawn to deliberately favor one party, we call it *gerrymandering*.

A central mathematical question is: **given a proposed map, is it unusual?** Of all the "reasonable" ways one could have drawn the districts, does this particular map produce extreme partisan outcomes?

To answer that, redistricting is abstracted as follows. Take the geographic region (a state) and represent it as a graph $G = (V, E)$:

- Each vertex represents a small geographic unit (a voting precinct).
- Two vertices are joined by an edge if the corresponding units are adjacent.
- Each vertex p carries a population weight w_p .

A **districting plan** is then a partition of V into I subsets (one per district). For the partition to count as a valid plan we typically require:

- **Contiguity:** each district induces a connected subgraph of G .
- **Population balance:** each district has roughly the same total population.

- Possibly other criteria: compactness, county preservation, Voting Rights Act compliance.

The set of valid plans is finite but astronomical. For a state like North Carolina with 2,671 precincts and 14 districts, the count dwarfs anything we could enumerate. So instead we **sample** from the state space.

The strategy: build an *ensemble* of plans drawn from a distribution that respects the legal/structural rules but is otherwise neutral. Then overlay actual voting data and ask whether the enacted plan looks typical or like a wild outlier. This methodology has been used as evidence in litigation in North Carolina, Pennsylvania, and elsewhere.

Concretely, ensembles are typically specified by a target distribution

$$\pi(x) \propto e^{-J(x)}$$

where x is a plan and J is a *score function* encoding hard and soft constraints. Sampling from π is the central computational task.

2. Why grid graphs?

Real states are messy: thousands of irregular vertices, populations that vary by orders of magnitude, geometry shaped by history and geography. That's great for doing real science, but terrible for understanding what an algorithm is *actually doing*.

Grid graphs are the standard testbed for a reason:

- An $m \times n$ grid has mn vertices and a clean, symmetric geometry.
- For very small instances we can sometimes enumerate all valid l -partitions, giving us **ground truth**.
- We can vary m , n , l systematically and watch how algorithm behavior changes as the state space grows.
- Population balance is easy to reason about: every vertex has weight 1, so each district should contain mn/l cells.

When we say "explore the state space of districting small grid graphs," we mean: look at all the ways to partition an $m \times n$ grid into l contiguous, balanced pieces, and study how Markov chains move through that space — and how our stratification pipeline organizes it.

A useful warm-up: try to enumerate by hand all the ways to partition a 4×4 grid into 4 contiguous, balanced regions of 4 cells each. You'll discover this is already nontrivial. Now imagine 6×6 into 4 districts, or 8×8 — the count explodes.

The `gmstrat` repo already has a grid-graph generator and ships with a `grid_graph_5_by_5.json` example, so you can start sampling on grids almost immediately.

3. Markov chain Monte Carlo, briefly

If you haven't seen MCMC before, here's the short version. Suppose you want to sample from a probability distribution π on a huge set X — in our case, X is the set of valid plans. You can't enumerate X , so you can't sample directly. Instead, you design a *Markov chain* — a random process that hops from one state to another — whose stationary distribution is π . Start at any valid state, run the chain for a long time, and the visited states will (in the limit) look like samples from π .

Two things you need to worry about:

- **Ergodicity / irreducibility:** can the chain reach every state from every other state? If not, you'll never see entire regions.
- **Mixing time:** how long until the distribution of the current state is close to π ? If the chain mixes slowly, your samples are correlated and biased toward where you started.

Both turn out to be hard for districting state spaces, which is why algorithm design here is an active research area.

A piece of vocabulary you'll see: *Metropolis–Hastings*. Given a proposal mechanism that suggests a next state x' from current state x , you accept x' with probability $\min(1, \pi(x')q(x' \rightarrow x) / (\pi(x)q(x \rightarrow x')))$, where q is the proposal density. If you accept, move; if you reject, stay. This guarantees the chain has stationary distribution π (assuming mild conditions), but only if you can compute the proposal probabilities — which is harder than it sounds for the chains we'll see.

4. The Markov chains

There are roughly three generations of Markov chains for sampling districting plans. We'll focus on the second and third.

4.1 Single-node Flip (the baseline)

The simplest proposal: pick a vertex on the boundary between two districts, and reassign it to the neighboring district. Check that the resulting plan is still valid; if so, accept (possibly with a Metropolis–Hastings correction).

Flip walks are easy to implement and easy to analyze. They are also notoriously slow: each step changes a single vertex, so getting from one shape to a meaningfully different shape takes enormous numbers of steps. Flip chains also have a bias toward "snaky," non-compact districts, because most contiguous partitions of a graph are weird-looking. We won't use Flip directly, but it's the foil that motivates everything else.

4.2 ReCom (Recombination)

ReCom was introduced by DeFord, Duchin, and Solomon (2021) and is the workhorse of modern redistricting analysis. The idea is to take much bigger steps. One iteration:

1. **Pick two adjacent districts** D_i and D_j .
2. **Merge** them into a single subgraph $G_{\{ij\}}$.
3. **Draw a uniformly random spanning tree** T of $G_{\{ij\}}$ (using Wilson's algorithm via loop-erased random walks).
4. **Find an edge in T** whose removal splits T into two subtrees with balanced populations. Cut that edge: the two resulting subtrees become the new D_i and D_j . If no balanced cut exists, reject and try again.

Why is this clever?

- Each step potentially redraws the entire boundary between two districts — these are big moves.
- Spanning trees on a connected subgraph guarantee contiguity by construction.
- The implicit distribution favors compact districts: tree-cuts on dense regions tend to balance more easily than cuts on stringy regions. Compactness emerges from the algorithm rather than being imposed by hand.
- Empirically, ReCom mixes far faster than Flip on real graphs.

Catch: vanilla ReCom's stationary distribution is the "spanning-tree" or "tree-count" measure, which is fine if that's what you want, but if your target π is something else, you have a problem. Computing forward and reverse proposal probabilities for ReCom is intractable because each proposal samples a fresh spanning tree, so you can't directly Metropolize ReCom. There are reversible variants — Metropolized Forest ReCom, multi-scale merge-split — that fix this by extending the state space from partitions to spanning forests, at the cost of complication.

4.3 Cycle Walk

Cycle Walk (DeFord, Herschlag, Mattingly, 2025) is the chain we'll mostly use. It's designed to address situations where ReCom-type chains struggle to sample from a *user-specified* target distribution — for instance, when you want to weight plans by policy criteria.

The trick is to operate on **spanning forests** rather than partitions directly. Instead of representing a state as a partition $\xi : V \rightarrow \{1, \dots, I\}$, you represent it as a spanning forest F of G with I trees. Each tree is the spanning tree of one district; forgetting tree structure recovers the partition. The state space is richer (one partition corresponds to many spanning forests), but this expansion makes transition probabilities tractable to compute.

One step of Cycle Walk:

1. Start with a spanning forest F whose trees define the current districts.
2. Add an edge e of G that is *not* in F . Adding e to F creates exactly one cycle C .
3. Remove a different edge e' of C , breaking the cycle. The result is a new spanning forest F' .
4. Accept or reject according to a Metropolis–Hastings rule that targets the desired distribution on partitions.

The cycle move is the elementary operation. It can change the partition (when e and e' lie in different trees) or just rearrange a tree (when they lie in the same tree). Composing many cycle moves explores the state space.

What makes Cycle Walk attractive:

- It is reversible by construction, so you can target arbitrary distributions via Metropolis–Hastings.
- It samples efficiently from distributions that have been hard for prior chains, including weighted distributions encoding policy preferences.
- The relevant transition probabilities are computable on the spanning-forest state space.

The tradeoff: each move is more local than ReCom's (one edge swap, not a whole boundary), so individual steps are cheaper but you may need more of them. In `gmstrat`, sampling is done by the Julia code in `sampling/`, adapted from `CycleWalk.jl`.

5. Why we need stratification, and how `gmstrat` does it

5.1 The motivation

Even with Cycle Walk, sampling has a fundamental problem: when the target distribution π puts most of its mass in one region of plan space, the chain spends most of its time there and rarely visits the tails. But the tails are exactly where we often want to look — to estimate rare events ("how often does this plan produce extreme partisan outcomes?") or to compare against atypical proposed maps.

This is the same problem that computational chemists faced fifty years ago when computing free-energy differences. Their solution, due to Torrie and Valleau (1977), is *umbrella sampling*: don't try to sample from π directly. Instead, decompose π into easier-to-sample pieces — *strata* — that each focus on a different region. Sample each stratum separately, then stitch the results back together. The modern formulation, EMUS (Eigenvector Method for Umbrella Sampling, Dinner et al. 2020), recasts this as a fixed-point problem on a stochastic matrix.

The catch: in chemistry the phase space is usually a subset of $\mathbb{R}^n$, where you have natural ways to define strata (e.g. intervals of a reaction coordinate). For redistricting plans, the phase space is **discrete and combinatorial**, with three nasty features:

1. Two plans can differ in only a single boundary precinct yet count as "distinct"; any useful notion of similarity must ignore inconsequential noise.
2. A plan is an *unordered* tuple of districts, so comparing two plans involves a district-matching problem (expensive in general).
3. Clustering plans directly is computationally heavy and obscures district-level structure (variation in one district is largely independent of variation in another).

The `gmstrat` pipeline (Wang, Herschlag, Yim, Mattingly, Gilbert, 2026) sidesteps these by working at the **district level first**, then lifting to plans.

5.2 The pipeline at a glance

The flow is:

ensemble of plans → **pool of districts** → **letters** → **words** → **strata (with POU)** → **stratum weights and flux**

Let me walk through each step.

5.3 Districts as vectors, with a population-weighted distance

Represent each district as a 0/1 indicator vector $v \in \{0,1\}^P$ over precincts, where $v_p = 1$ iff precinct p is in the district. Define a **population-weighted, capped ℓ^1 distance** between two districts:

$$d(v, v') = \min \left(d_{\max}, \sum_{p=1}^P |v_p - v'_p| w_p \right).$$

The cap $d_{\max}$ prevents disjoint districts from being treated as "infinitely far apart" — that just adds noise. A standard choice is $d_{\max} = (2/I) \cdot \sum w_p$ (twice the ideal district population). The plan-level distance $D(x, x')$ is then the minimum-cost matching between districts under d .

5.4 Letters: representative districts via hierarchical clustering

Pool all the districts that appeared across an ensemble of M sampled plans into a set D of size up to $M \cdot I$ (with duplicates removed). Cluster D using **HccUltraFit** (Yim & Gilbert, 2024), a hierarchical correlation clustering algorithm with provable guarantees on how well the dendrogram's ultrametric approximates the original distances. Standard linkage methods (single, complete, average) lack this consistency.

The dendrogram is then *cut* at a chosen height to produce K clusters. Each cluster D_k has a **letter** ℓ_k — a representative district — defined as the centroid

$$\ell_k = \arg \min_{\ell \in \{0,1\}^P} \sum_{v \in D_k} d(\ell, v).$$

Under the weighted ℓ^1 distance, this reduces to a precinct-wise weighted majority vote: precinct p is in the letter iff at least half the districts in the cluster contain it.

To choose K , use the elbow method on the within-cluster dispersion $L(K) = \sum_k \sum_{v \in D_k} d(v, \ell_k)$. On Connecticut (5 districts) the elbow sits around $K = 34$; on North Carolina (14 districts) it's around $K = 129$.

5.5 Words: representative plans

A **word** is an unordered I -tuple of letters — a representative plan. Words live in the same space as plans but are vastly compressed: at most $\sim K^I / I!$ many of them, but only a tiny fraction correspond to anything resembling a real plan.

For each observed plan x , find its L nearest words under D using **beam search** (Algorithm 1 in the paper). Concretely: build the word one district at a time, at each step keeping only the L best partial

candidates and ruling out repeated letters. This costs $O(K \log L + K \log K)$ per plan and is exact for the L -nearest-neighbors problem.

The representative word set S is the union of these L -neighborhoods across the ensemble. You should picture it as a bipartite graph (Figure 3 in the paper): plans on the left, words on the right, each plan connected to its L nearest words.

5.6 The Partition of Unity (POU)

Now we softly assign each plan to its nearest words. Define an unnormalized kernel

$$\psi(x; s) = \begin{cases} \exp(-T\kappa(D(x, s))) & \text{if } s \text{ is among the } L \text{ nearest words of } x \\ 0 & \text{otherwise} \end{cases}$$

and normalize across strata:

$$\phi_s(x) = \frac{\psi(x; s)}{\sum_{s' \in S} \psi(x; s')}, \quad \sum_{s \in S} \phi_s(x) = 1.$$

T is a temperature; κ is a function that maps distances (in units of population) into a sensible range. Tuning (T, κ) controls the coverage/overlap tradeoff: too peaked and adjacent strata can't communicate; too smeared and EMUS-style fixed-point iterations become unstable.

5.7 Stratum weights and the flux matrix

Define stratum weights and a flux matrix:

$$z_s := \mathbb{E}_\pi[\phi_s], \quad F_{s,s'} := \mathbb{E}_{\pi_s}[\phi_{s'}] = \frac{1}{z_s} \mathbb{E}_\pi[\phi_s \phi_{s'}],$$

where π_s is the tilted stratum distribution $d\pi_s/d\pi = (1/z_s)\phi_s$. Two important properties:

- **F is row-stochastic.** Each row sums to 1 because the $\phi_{\{s'\}}(x)$ sum to 1 at every point.
- **z is the stationary distribution of F.** That is, $z^\top F = z^\top$. You can verify this by direct computation; it's the analog of the EMUS fixed-point equation.

So F can be read as the transition kernel of a Markov chain on **strata**, and z tells you how much of the target distribution lives in each stratum. The flux $F_{\{s, s'\}}$ measures how often, under π_s , mass spills over into stratum s' — this is the bottleneck quantity that controls how well stratified sampling will mix.

5.8 Pruning redundant strata

When you have a lot of overlap, S can be highly redundant — two words may differ only in a tiny local letter substitution. Pruning is a **set-cover** problem: keep the smallest sub-collection $S_0 \subseteq S$ that still covers every observed plan within some distance threshold η . Set-cover is NP-hard, so we use the standard greedy heuristic (Algorithm 2 in the paper), which gives a $(\log|S| + O(1))$ -approximation. The pruned strata are far more interpretable and correspond to genuinely distinct large-scale plan motifs.

5.9 Putting it together

The output of the pipeline is:

- An **alphabet** of K representative districts (letters),
- A **dictionary** S of representative plans (words),
- A **partition of unity** $\{\phi_s\}$ that softly assigns plans to strata,
- **Stratum weights** z_s and a **flux matrix** $F_{\{s,s'\}}$ that summarize the coarse dynamics on words.

This is the foundation for actual stratified sampling, which would run S separate Cycle Walk chains targeting tilted distributions $\pi_s \propto \exp(-J + \phi_s)$ and combine them via fixed-point iteration to recover $E\pi[f]$ for an observable f . The current paper sets up the stratification; running the full stratified sampling is one of the things we may do this summer.

6. The project

Goal. Use `gmstrat`, driven by Cycle Walk (and possibly ReCom for comparison), to explore the state space of partitions of an $m \times n$ grid graph into l contiguous, population-balanced districts.

Why grids, again: we can vary m , n , l systematically; on small grids we can compute or near-enumerate ground truth; the geometry is clean enough that anything weird in the pipeline's output reflects the algorithm rather than messy real-world data.

The pipeline was designed for and tested on Connecticut and North Carolina. **Grid graphs are a new domain for it**, and that's where the project starts.

A non-exhaustive list of questions we'd like to answer:

1. **What do letters look like on a grid?** Letters are representative district shapes. On a grid we can ask: are they roughly rectangular? Do they break the symmetry of the grid? How does the

alphabet size K (chosen by the elbow method) scale with m , n , l ?

2. **What do words look like?** Real plans have hundreds of millions of possibilities. On small grids, how many "motifs" are there really? Does the pruned word set S_0 have a clean geometric interpretation (e.g. "horizontal stripes" vs "vertical stripes" vs "quadrants" on a 4×4 with $l=4$)?
3. **How does coverage scale?** As (m, n, l) grow, how large does L need to be so that every plan has at least one nearby word? At what point does the bipartite plan $\leftrightarrow$ word graph become genuinely sparse?
4. **What does the flux matrix tell us?** F encodes the coarse dynamics on strata. On grids small enough to interpret by eye, does F 's structure match geometric intuition — e.g., are nearby words (small symmetric-difference distance) connected by high flux?
5. **Ground truth comparisons.** For tiny grids where we can enumerate plans, how does the pipeline's z compare to the actual mass π puts in each stratum? This is a sanity check the Connecticut/NC examples can't provide.
6. **Cycle Walk vs. ReCom.** Both chains, run on the same grid with comparable target distributions, should produce ensembles that lead to similar pipeline outputs. Do they? Where do they diverge?
7. **(Stretch) Actual stratified sampling.** Take the strata `gmstrat` produces, run S parallel Cycle Walk chains targeting tilted distributions π_s , and combine via fixed-point iteration to estimate an observable. Compare variance to vanilla Cycle Walk.

These questions are interrelated, and part of your job will be to figure out — over the first couple of weeks — which is the most tractable starting point. We'll narrow down together.

7. Suggested first steps

1. **Read the references.** Roughly in this order: the DeFord–Duchin–Solomon ReCom paper (introduction and §1–2 carefully), then the Cycle Walk paper (introduction and §2), then the `gmstrat` paper (the whole thing). Don't try to absorb everything; aim for the lay of the land.
2. **Hand-enumerate** all valid 2-district balanced partitions of a 4×4 grid. Then 4-district partitions. This calibrates intuition for the state space.
3. **Get the repo running.** Clone `gmstrat`, set up the conda environment and Julia 1.11+, and run the demo notebook. The repo includes a `grid_graph_5_by_5.json`, so you can sample a grid out of the box:

```
./sampling/run.sh --map-file data/graph/grid_graph_5_by_5.json \  
--output-file local/output/grid5x5/atlas.jsonl.gz \  
--cycle-walk-steps 1e5 \  
--pop-dev 0.1
```

4. **Run the full pipeline on a small grid.** Generate an ensemble, run hierarchical clustering on districts, build words, compute the POU and flux. Get visualizations working — recreate analogs of Figures 7–12 from the paper, but for grids.
5. **Pick a question.** Vary m , n , or l along one axis and see what happens.

We'll meet weekly (more often at the start). Bring questions and confusions — getting confused about something specific is one of the best signs that you're learning.

8. References

The four most important things to read:

1. **Wang, Herschlag, Yim, Mattingly, Gilbert (2026).** *Towards Stratified Sampling for Redistricting Plans*. Our group's paper describing the pipeline you'll be using. The authoritative reference for the methodology.
2. **DeFord, Duchin, Solomon (2021).** *Recombination: A family of Markov chains for redistricting*. Harvard Data Science Review 3(1). The canonical introduction to ReCom; written for non-specialists. <https://hdr.mitpress.mit.edu/pub/1ds8ptxu>
3. **DeFord, Herschlag, Mattingly (2025).** *A Cycle Walk for Sampling Measures on Spanning Forests for Redistricting*. arXiv:2509.08629. Read the introduction and §2 carefully. <https://arxiv.org/abs/2509.08629>
4. **The gmstrat repository.** <https://github.com/zijian-w/gmstrat> — code, demo notebook, and grid examples.

Useful background and follow-up:

- **Bangia et al. (2017).** *Redistricting: Drawing the Line*. arXiv:1704.03360. An accessible early treatment of MCMC for redistricting in North Carolina.
- **Carter, Herschlag, Hunter, Mattingly (2019).** *A merge-split proposal for reversible MCMC sampling of redistricting plans*. arXiv:1911.01503. The reversible variant of ReCom.
- **Autry, Carter, Herschlag, Hunter, Mattingly (2023).** *Metropolized Forest Recombination for Monte Carlo Sampling of Graph Partitions*. SIAM J. Appl. Math.

- **Dinner, Thiede, Van Koten, Weare (2020).** *Stratification as a general variance reduction method for Markov chain Monte Carlo.* The modern formulation of EMUS — the umbrella-sampling theory underlying the pipeline.
- **Torrie & Valleau (1977).** *Nonphysical sampling distributions in Monte Carlo free-energy estimation: Umbrella sampling.* J. Comp. Phys. The original umbrella-sampling paper.
- **Yim & Gilbert (2024).** *Fitting trees to ℓ^1 -hyperbolic distances.* arXiv:2409.01010. The HccUltraFit clustering algorithm used in the pipeline.

Questions: ask in our Slack or email me directly.